

CLASS 06 / SEP 29 / U04

ARRAYS 1

LINE · GRID · CUBE · CURVE

ONE MODULE, MANY POSITIONS

2.1 DUE TONIGHT

3.2 ISSUED TODAY / 2.2 PRINT CHECK TODAY

AN ARRAY IS A LIST OF POSITIONS

Not a command that duplicates things. A list you compute, then place your module at.

INDEX → POSITION

$i = 0 \rightarrow p = \text{start}$

$i = 1 \rightarrow p = \text{start} + \text{step}$

$i = 2 \rightarrow p = \text{start} + 2 \times \text{step}$

$p[i] = \text{start} + i \times \text{step}$

THREE THINGS TO READ

THE LIST — what is in the collection.

THE COUNT — how many there are.

THE INDEX — where each one sits in the order.

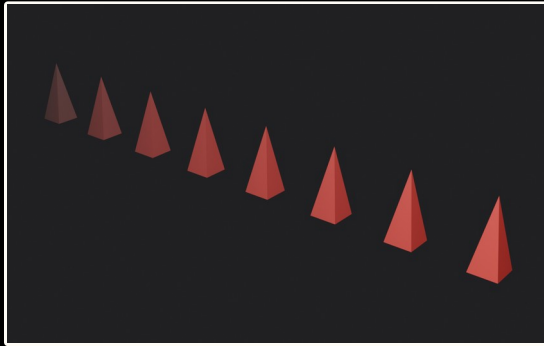
The index is the only new idea. It is a counter that tells each copy how far along it is — and therefore where it goes.

Your 3.1 module is the component. On Sheet 1 it does not change — only where it goes.

TODAY: ONE LOOP, FOUR WAYS

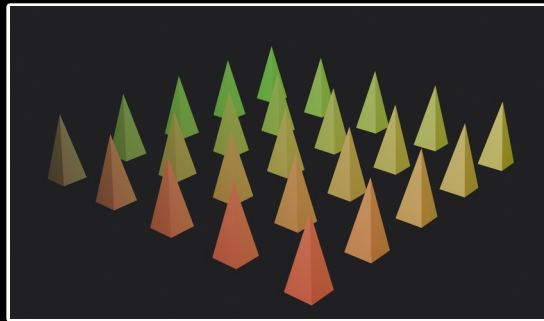
Class file Arrays Part 1, scenes 01-04. Colour is position: X red, Y green, Z blue.

01 LINE



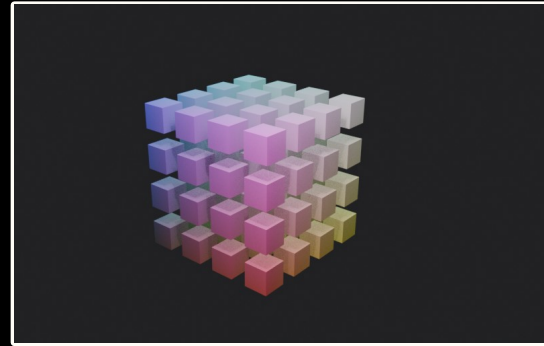
i

02 GRID



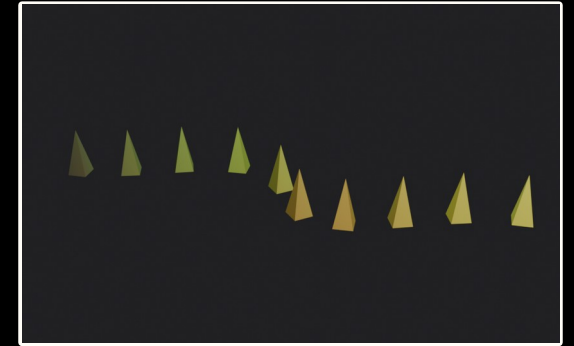
i, j

03 CUBE



i, j, k

04 CURVE



i along a curve

Line, grid and cube add one counter at a time. The curve changes where a position comes from. No rules yet: every copy is treated the same. Patterns start next week.

EVERY STEP IS THE SAME LOOP. ONLY THE COUNTERS AND THE SOURCE OF POSITION CHANGE.

THE LOOP, WRITTEN OUT

Scene 01 — Linear loop, as a script you can read aloud.

```
count    = 8
spacing  = 0.8
points   = []                # the list starts empty

for i in range(count):      # i = 0, 1, 2 ... 7
    x = i * spacing         # position from the index
    points.append((x, 0, 0)) # one more item

place(module, points)      # a copy at every point
```

READ IT AS A SENTENCE

Start with an empty list.

Count times: work out x from i, and add that point to the list. Each pass, i is one bigger.

Then put the module at every point.

In the class file: for → Repeat Zone · i → Iteration · append → Join Geometry · place → Instance on Points

i IS THE INDEX. EVERYTHING ELSE IN THE LOOP IS ARITHMETIC ON IT.

READ IT SMALL FIRST

The loop, and a spreadsheet open beside it.

WHAT THE LOOP DOES

Everything inside the loop runs once per pass. Whatever goes in comes back with one more item, once for every step of the count.

HOW TO CHECK IT

Set Count to 3 and read the spreadsheet: three rows, three positions.

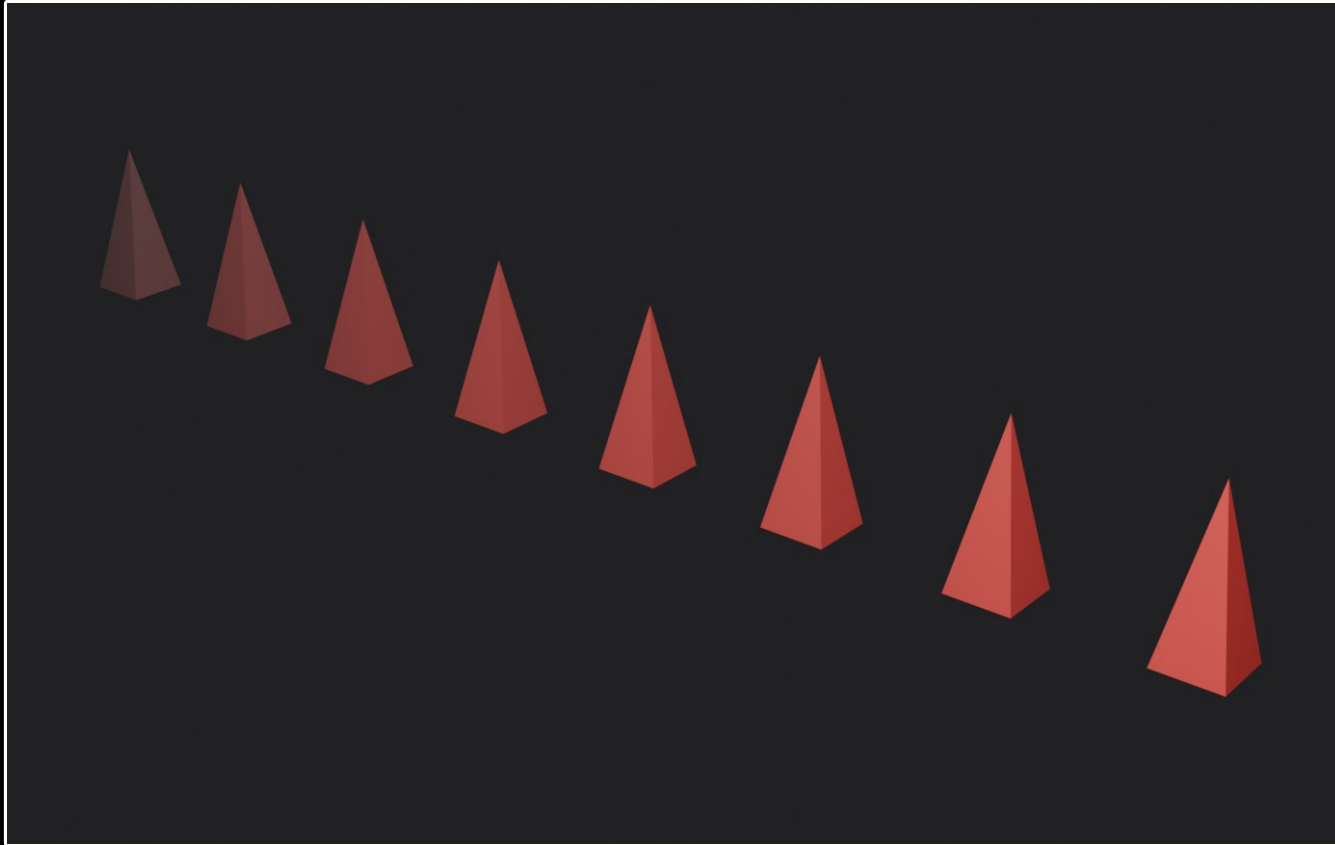
Then 4. Then 5. Say the last X before you look.

A list you cannot inspect is a list you cannot debug. There is no minimum count.

BUILD IT AT A COUNT YOU CAN COUNT. SCALE UP ONLY ONCE IT IS RIGHT.

01 · LINE

Scene 01 — Linear loop · Count 8 · Spacing 0.8



READ THE ROW

i	0	1	2	...	7
x	0.0	0.8	1.6	...	5.6

count = 8

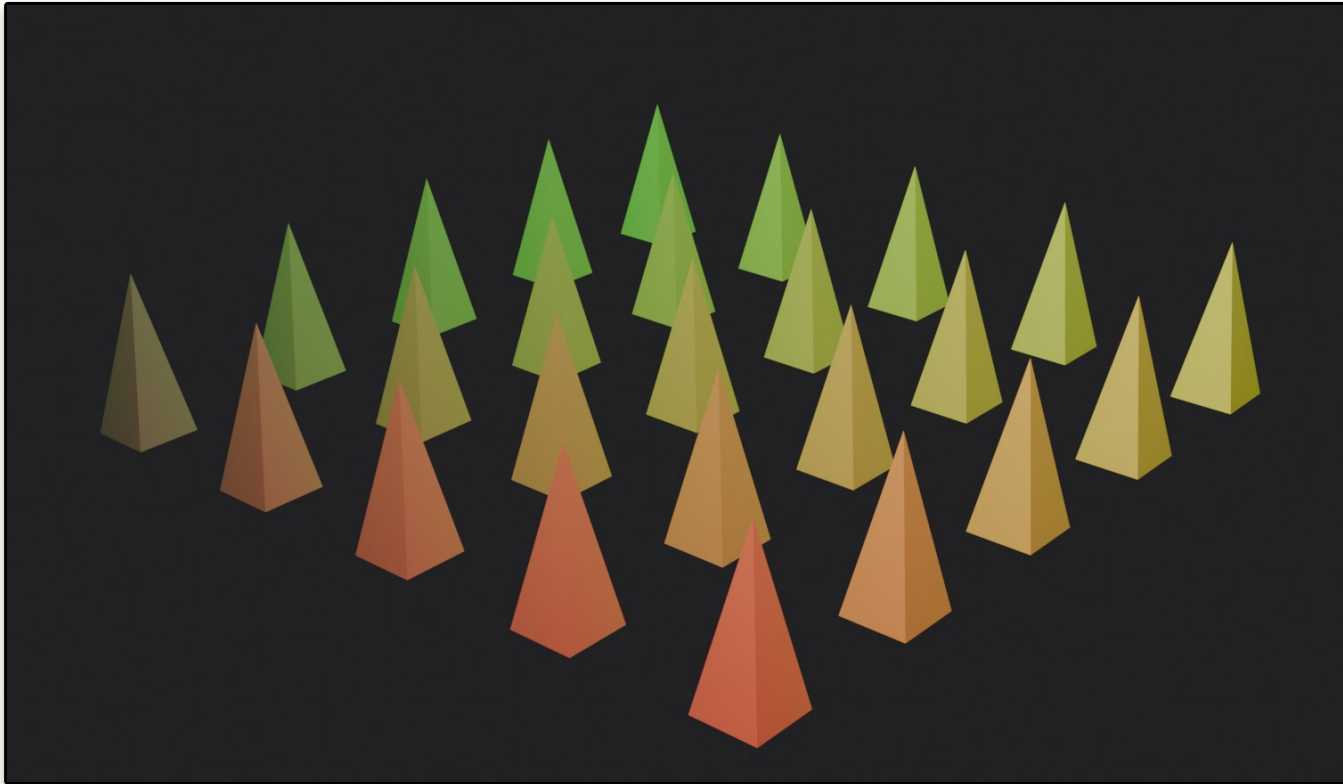
last = $7 \times 0.8 = 5.6$

Colour is position: $X \rightarrow \text{red}$. The darkest copy is $i = 0$, the brightest $i = 7$.

THE LAST COPY IS AT $7 \times \text{SPACING}$, NOT 8. COUNTING STARTS AT ZERO.

02 · GRID — A LOOP INSIDE A LOOP

Scene 02 — Nested grid · Rows 5 · Columns 5 · Spacing 0.8



TWO COUNTERS

```
for j in range(rows):  
    for i in range(columns):  
        place(i*dx, j*dy, 0)
```

$$5 \times 5 = 25$$

i runs along the row; j picks which row. Red grows with X, green with Y.

THE INNER LOOP MAKES A ROW. THE OUTER LOOP COLLECTS THE ROWS.

INDEX ORDER

Which counter runs inside decides the order. Label it on the sheet.

A 4 × 3 GRID, i INSIDE j

8	9	10	11	$j = 2$
4	5	6	7	$j = 1$
0	1	2	3	$j = 0$

$i \rightarrow 0 \cdot 1 \cdot 2 \cdot 3$

THE FLAT INDEX

$$\text{2D: index} = i + j \times nx$$

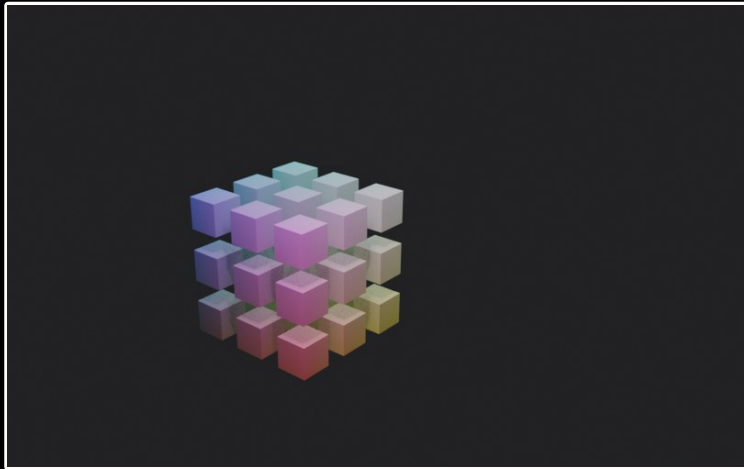
$$\text{3D: index} = i + j \times nx + k \times nx \times ny$$

Swap the loops and the same positions get different numbers.
The picture looks the same; any rule that reads the index — next week's if/Switch — does not.

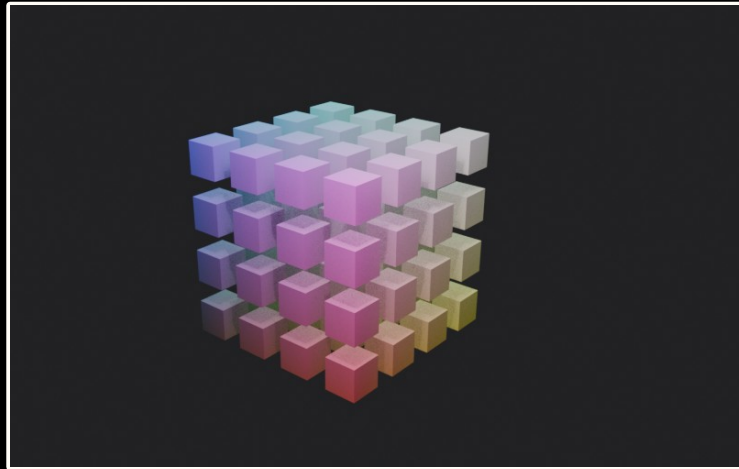
SAY IT OUT LOUD: WHICH WAY DOES i RUN, AND WHAT HAPPENS IF YOU SWAP THE LOOPS?

03 · CUBE — THE GRID, REPEATED IN Z

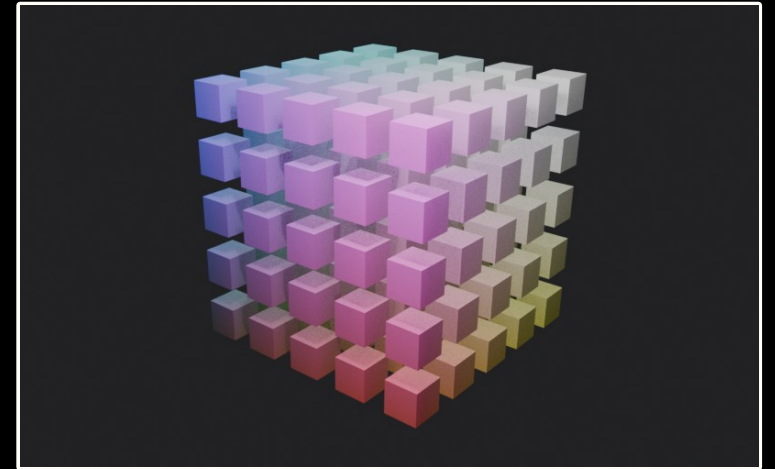
Scene 03 — 3D array · Spacing 1.25 · Cube Size 0.75 · one camera for all three



COUNT 3 → $3 \times 3 \times 3 = 27$



COUNT 4 → $4 \times 4 \times 4 = 64$



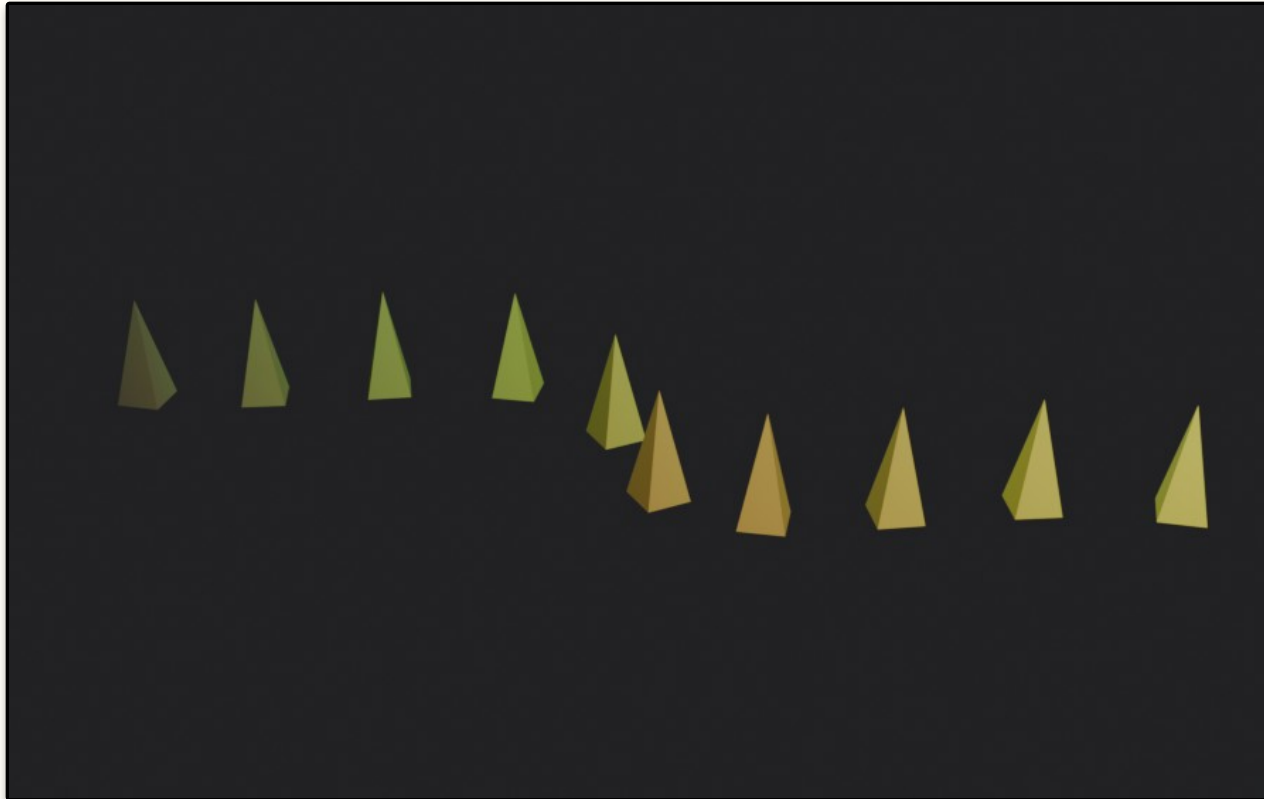
COUNT 5 → $5 \times 5 \times 5 = 125$

```
for k in range(n): for j in range(n): for i in range(n): place(i*dx, j*dy, k*dz)
```

PREDICT THE TOTAL BEFORE YOU LOOK: 27, 64, 125. X, Y AND Z TOGETHER MAKE THE RGB CUBE.

04 · CURVE — THE ROW BENDS

Scene 04 — Along a curve · Spacing 0.8 · the same module as the line. Edit the curve; the row follows.



WHERE IS COPY i?

```
count = int(length / spacing) + 1
for i in range(count):
    t = i / (count - 1) # 0 → 1
    p = point_on(curve, t)
    d = direction_on(curve, t)
    place(module, p, facing=d)
```

The line said $x = i \times \text{spacing}$. Here the curve answers the same question, and also says which way to face.

SAME LOOP AS THE LINE. THE CURVE GIVES THE POSITION INSTEAD OF $i \times \text{SPACING}$.

THE EVOLUTION

Line to curve, one change at a time.

01 LINE

One counter, i . Position = $i \times \text{spacing}$.

02 GRID

i inside j . Total = rows $\times$ columns.

03 CUBE

i inside j inside k . Total = count cubed.

04 CURVE

One counter again — but the curve gives the position and the direction.

NEXT WEEK

Patterns: a rule that reads the index. First example — a brick wall, every other course shifted.

ONE LOOP. EACH STEP ADDS A COUNTER OR CHANGES WHERE THE POSITION COMES FROM.

TODAY, STEP BY STEP

Plain repetition: every copy treated the same. The curve is in-class practice; Sheet 1 is your module, then 1D, 2D, 3D.

01

1D: build a small linear array with Repeat Input and Repeat Output. Identify list, count and index.

02

2D: nest repetition into a grid. Identify rows and columns; predict the total count.

03

3D: repeat the grid in Z for a cube array. Predict Count cubed; distinguish instances from a sampled volume.

04

Curve: bend the row. Sample a curve at even spacing and place your module at each point, turned to follow the curve's direction. The count comes from $\text{curve length} \div \text{spacing}$.

2.2 PRINT CHECK — BEFORE YOU QUEUE

Check geometry, units, orientation and the sliced preview before queueing.

GEOMETRY

Closed and solid. Every edge shared by two faces, no stray pieces.

UNITS

Real millimetres. Within 60 mm on each axis.

ORIENTATION

No supports. Orientation is your lever; change the geometry if you must.

SLICED PREVIEW

Read it layer by layer before queueing. Stay within the 200 g allocation.

2.2: HAND IN THE PRINT IN CLASS, WEEK 8 — OCT 13. DIGITAL FILES AND NOTES BY 11:59 PM THAT NIGHT.

ISSUED TODAY

3.2 — ARRAYS

DUE WEEK 10 — Oct 27 / 10%

Three 17×11 sheets, submitted together as one assignment. Sheet 1 is today; Sheets 2 and 3 are next week's class.

SHEET 1 • TODAY

MODULE, 1D, 2D, 3D

Your 3.1 module alone, then in a row, a grid and layers. Label counts, spacing, axes and totals.

SHEET 2 • WEEK 7

FOUR FAÇADE PATTERNS

Your module in a 2D array, changed by if/Switch conditions. Each rule and its outcome captioned.

SHEET 3 • WEEK 7

MATH SHAPES A VOLUME

Layered cubes, height driven by a function of X and Y. One isometric, formula and settings stated.

REQUIREMENTS — 3.2

Computational understanding 30% · Visual communication 30% · Technical execution 25% · Exploration 15%.

01

Submit exactly three 17 x 11 inch landscape sheets together as one assignment.

02

Sheet 1 — four diagrams: your 3.1 module alone, your module in a 1D array, your module in a 2D array, and your module in a 3D array. Use the same module and label counts, spacing and axes.

03

Sheet 2 — four façade-pattern diagrams: use your module in a 2D array and if/Switch conditions to generate four different patterns. Each diagram must identify its condition and resulting modification.

04

Sheet 3 — volumetric exploration: use cubes in a 3D array and present an isometric view. Drive vertical variation with a mathematical function of both X and Y, such as sine or cosine. Explore the resulting spatial form; optionally combine it with Switch conditions to generate voids.

05

Reuse the 3.1 panel definition on Sheets 1–2. Sheet 2 may transform or omit selected instances to make a pattern. Use cubes, rather than the panel, for the volumetric exercise on Sheet 3.

06

Present simple diagrams using one consistent view and material for comparable studies. Keep lighting consistent; separate polished render setups for the four façade patterns are not required. Choose an appropriate view for each dimensional array and use isometric for the cube volume.

07

Build repetition from basic nodes and begin with a count you can inspect; there is no minimum element count.

08

Use materials from the instructor-provided library or create your own. Record which materials you used and any edits; package required textures. Material choice must keep array spacing, depth and repetition readable. The library will be posted before this assignment is due; creating your own remains an equally valid route.

NOW — SHEET 1

Predict every total before you look.

✓

A row: Count and Spacing exposed, spreadsheet open, the last X said out loud

✓

The row nested into a grid: rows $\times$ columns predicted first

✓

The grid repeated in Z — 3, 4 and 5 per axis compared

✓

Your row bent onto a curve: edit the curve and watch the row follow

✓

2.1 uploaded by 11:59 PM tonight; 2.2 print checked before you queue it

A COUNT YOU CAN COUNT. NOT A THOUSAND OF ANYTHING, YET.

BEFORE NEXT CLASS

NEXT

Week 7 — Oct 6: patterns — a brick wall with every other course shifted, if/Switch façade patterns, and a cube volume from X and Y (Sheets 2 and 3). Bring Sheet 1's graph working; we build on it.

DUE

Nothing graded next week (studio reviews). 2.2 print in class Oct 13. 3.2, all three sheets: Oct 27.

BUILD

Finish Sheet 1: module alone, then 1D, 2D and 3D — counts, spacing, axes and totals labelled.

PRINT

Fix anything today's check flagged, then queue 2.2.